INSTITUTO FEDERAL
Goiás
Câmpus Goiânia

Instituto Federal de Educação, Ciência e Tecnologia de Goiás

Pró-Reitoria de Ensino

Câmpus Goiânia

Especialização em Inteligência Artificial Aplicada

Igor Coutinho Ferreira Nobre

Otimização da Dieta de Vacas em Lactação pelo Método Simplex Integrado ao RuFaS

Módulo de otimização, efeito econômico e análise de sensibilidade

Relatório de Disciplina

Professor: Dr. Eduardo Noronha de Andrade Freitas
Disciplina: Modelagem e Otimização Aplicada

Goiânia, 2026.

1 INTRODUÇÃO E MODELAGEM DO PROBLEMA

Este relatório foi elaborado para a disciplina Modelagem e Otimização Aplicada, do curso de Especialização em Inteligência Artificial Aplicada do IFG, ministrada pelo Prof. Dr. Eduardo Noronha de Andrade Freitas. O projeto final tem o enunciado: *"Desenvolvimento de módulo de otimização da dieta integrada ao RuFaS com avaliação de efeito econômico, e também análise de sensibilidade."*

O Ruminant Farm Systems (RuFaS) é um simulador de fazendas leiteiras, de código aberto, em Python (Ruminant Farm Systems, 2026). A cada 45 dias ele calcula, para o animal médio de cada lote, as exigências nutricionais pelo padrão National Academies of Sciences, Engineering, and Medicine (NASEM) (2021) (National Academies of Sciences, Engineering, and Medicine (NASEM), 2021) e formula a dieta de menor custo. O estudo de caso é a fazenda sintética Campo Dourado (GO), disponibilizada em sala pelo professor, simulada de 2015 a 2019, com 41 formulações do lote de vacas em lactação. Todo o material está no fork do autor (Nobre, 2026), ramo `simplex`: os arquivos da fazenda na pasta `GO_campo_dourado/`, o otimizador em `RUFAS/biophysical/animal/ration/` e o painel de sensibilidade em `analise_sensibilidade/`.

1.1 O que foi construído

O RuFaS já possui um otimizador, o método não linear Sequential Least Squares Programming (SLSQP) da biblioteca scipy. Neste trabalho acrescentei um segundo solver, construído com a biblioteca PuLP (COIN-OR Foundation, 2026) mas resolvido pelo Simplex, que grava o tableau ótimo de cada formulação para a análise de sensibilidade. Segui três regras: o código do SLSQP não foi alterado e continua sendo o padrão; o resolvedor é escolhido só na linha de comando (`-r simplex`); e os arquivos da fazenda ficam intocados. Os capítulos seguintes mostram o Simplex no problema, a análise de sensibilidade e os resultados contra o otimizador existente.

1.2 Modelagem do problema

1.2.1 Variáveis de decisão

x_j é a quantidade do alimento j na dieta, em kg de matéria seca (MS) por vaca por dia, $j = 1,\ldots,6$ (Tabela 1, do arquivo `GO_campo_dourado/inputs_resolvidos/feed.json`).

Tabela 1 – *Alimentos do lote de lactação da fazenda Campo Dourado.*

j	Alimento	Categoria	c_j (US$/kg MS)	Receita (% da MS)
1	Silagem de milho	Forragem	0,16	30
2	Silagem de leguminosa	Forragem	0,18	18
3	Milho grão moído	Concentrado	0,19	24
4	Farelo de soja 48 %	Concentrado	0,41	16
5	Caroço de algodão	Concentrado	0,36	9
6	Núcleo mineral	Mineral	0,80	3

Fonte: Arquivo `GO_campo_dourado/inputs_resolvidos/feed.json`.

1.2.2 Função objetivo

Minimizar o custo diário da dieta de uma vaca em lactação:

$$\text{Min } Z = 0{,}16\,x_1 + 0{,}18\,x_2 + 0{,}19\,x_3 + 0{,}41\,x_4 + 0{,}36\,x_5 + 0{,}80\,x_6 \quad \text{[US\$/vaca/dia]} \tag{1}$$

1.2.3 Restrições

Tabela 2 – *Modelo de PL da dieta do lote de lactação (coeficientes por kg de MS).*

Restrição	x_1	x_2	x_3	x_4	x_5	x_6		b_i
R1 energia líquida (Mcal)	1,75	1,54	2,06	2,36	1,87	0	$\geq$	45,5
R2 proteína metab. mín. (g)	77,8	92,1	96,8	236,5	102,6	4,7	$\geq$	2 575
R3 proteína metab. máx. (g)	77,8	92,1	96,8	236,5	102,6	4,7	$\leq$	3 863
R4 cálcio absorvível (g)	0,71	3,76	0,21	2,40	1,04	146,6	$\geq$	65,2
R5 fósforo absorvível (g)	1,48	2,25	2,16	5,17	4,35	18,56	$\geq$	61,7
R6 FDN $\geq$ 25 % da MS	15,93	18,20	−15,24	−13,94	25,58	−24,86	$\geq$	0
R7 FDN $\leq$ 45 % da MS	4,07	1,80	35,24	33,94	−5,58	44,86	$\geq$	0
R8 FDN de forragem $\geq$ 15 %	25,93	28,20	−15	−15	−15	−15	$\geq$	0
R9 gordura $\leq$ 7 % da MS	4,08	4,13	3,15	5,19	−11,62	7,00	$\geq$	0
R10 consumo de MS mín. (kg)	1	1	1	1	1	1	$\geq$	17,0
R11 consumo de MS máx. (kg)	1	1	1	1	1	1	$\leq$	31,7
custo c_j (US$/kg MS)	0,16	0,18	0,19	0,41	0,36	0,80	min	

Fonte: Elaborado pelo autor a partir dos arquivos da fazenda e das saídas do RuFaS.

As exigências b_i vêm do RuFaS para a vaca média do lote (635 kg, 40 kg de leite/dia) e os coeficientes a_{ij} da tabela de composição NASEM do simulador. Fibra

e gordura são exigidas como fração da MS; como $\sum_j x_j > 0$, a razão vira restrição linear:

$$\frac{\sum_j a_{ij} x_j}{\sum_j x_j} \geq k \iff \sum_j (a_{ij} - k)\, x_j \geq 0. \tag{2}$$

A Tabela 2 traz o modelo: 6 variáveis, 11 restrições e os limites por alimento. Na otimização livre, $0 \leq x_j \leq 100$; no modo "receita do usuário", em uso na fazenda (`user_input = true`), cada alimento fica a $\pm 10\,\%$ da participação ρ_j da Tabela 1: $0{,}9\,\rho_j \cdot 1{,}1\,\mathrm{CMS} \leq x_j \leq 1{,}1\,\rho_j \cdot 1{,}1\,\mathrm{CMS}$, com CMS o consumo de MS estimado.

Nas linhas R1 a R3 há uma ressalva: no NASEM a energia e a proteína que um alimento entrega dependem da dieta inteira, então os coeficientes valem em torno da dieta praticada e, dentro do RuFaS, essas restrições são funções não lineares de x.

2 O SIMPLEX NO PROBLEMA

2.1 Do não linear ao linear

No RuFaS as restrições de energia e proteína são funções Python que recebem o vetor x inteiro; a função `_linearize` do arquivo novo `RUFAS/biophysical/animal/ration/simplex_optimizer.py` (Código 1) é o que as transforma em restrições lineares.

Código 1 Linearização de uma restrição (`simplex_optimizer.py`).

```
    def _linearize(name: str, fun: Callable[..., float], ration_config: Any, x_k:
        npt.NDArray[np.float64]):
        n = len(x_k)
        def g(z: npt.NDArray[np.float64]) -> float:
            return float(fun(np.asarray(z, dtype=float), ration_config))
        unit = np.eye(n)
        if name in RATIO_CONSTRAINTS and x_k.sum() > 0:  # razao: sum (a_j - k) x_j
             >= 0
            a = np.array([g(unit[j]) for j in range(n)])
            if np.isclose(g(2.0 * x_k), g(x_k), rtol=1e-7, atol=1e-9):
                return a, 0.0, "razao"
        if name in LINEAR_CONSTRAINTS:  # affine: recover the coefficients exactly
            g0 = g(np.zeros(n))
            a = np.array([g(unit[j]) - g0 for j in range(n)])
            if np.isclose(g(x_k), a @ x_k + g0, rtol=1e-7, atol=1e-9):
                return a, -g0, "linear"
        gk, grad = g(x_k), np.zeros(n)  # tangent plane by finite differences
        for j in range(n):
            h = 1e-5 * max(1.0, abs(float(x_k[j])))
            fwd, bwd = x_k.copy(), x_k.copy()
            fwd[j] += h
            bwd[j] -= h
            grad[j] = (g(fwd) - g(bwd)) / (2.0 * h) if bwd[j] >= 0.0 else (g(fwd) -
                 gk) / h
        return grad, float(grad @ x_k - gk), "linearizada"
```

A linearização por partes exige não linearidade separável, $f(x) = \sum_j f_j(x_j)$, e não é o caso: a energia da silagem de milho muda conforme a quantidade de milho grão na dieta. A solução que adotei foi a Programação Linear Sequencial

(PLS): linearizar as restrições em torno da dieta corrente x_k, resolver o PL pelo Simplex, mover a dieta para a solução e repetir até parar de mudar. O Código 1 trata cada restrição $g_i(x) \geq 0$ de três formas: **lineares** (cálcio, fósforo, consumo), com coeficientes recuperados nos vetores unitários; **de razão** (FDN, FDN de forragem, gordura), pela reescrita exata da Seção 1.2; e **não lineares** (energia, proteína), pelo plano tangente em x_k, $\nabla g(x_k)^\top x \geq \nabla g(x_k)^\top x_k - g(x_k)$.

Cada PL é resolvido numa região de confiança $x_k \pm r$; a nova dieta é aceita se a função de mérito (custo mais penalidade por violação das restrições originais) não piora, senão r cai pela metade. Ao final o PL é resolvido de novo na dieta final com os limites reais, para que o tableau exportado seja o dessa dieta, e a dieta devolvida ao RuFaS satisfaz as restrições originais. No caso base cada formulação precisou de duas a três iterações e de zero a cinco pivôs.

2.2 Modelagem com PuLP e Simplex primal

Cada PL é modelado com o PuLP e resolvido pelo Simplex primal do resolvedor HiGHS, com o pré-processamento desligado para que a contagem de iterações seja a do Simplex (Código 2). Limites inferiores entram por mudança de variável ($x_j = L_j + x'_j$) e superiores como linhas $x_j \leq U_j$. Como nenhuma biblioteca expõe o tableau, ele é reconstruído da base ótima devolvida: $B^{-1}[A \mid S]$, com folga por restrição $\leq$, excesso por restrição $\geq$ e a linha Z com os custos reduzidos.

Código 2 Modelagem com PuLP e chamada do Simplex primal do HiGHS (`simplex_optimizer.py`).

```
    def _solver(self) -> pulp.LpSolver:
        return pulp.HiGHS(msg=False, solver="simplex", simplex_strategy=self.
            simplex_strategy, presolve="off")
    def solve(self, lp: LinearProgram) -> SimplexResult:
        n = len(lp.c)
        c, lower = np.asarray(lp.c, dtype=float), np.asarray(lp.lower, dtype=float)
        A, b, senses, names = self._rows_with_upper_bounds(lp)
        b_shift = b - A @ lower  # x = lower + x', x' >= 0
        # PuLP sorts variables by name: zero-padded names keep the decision-vector
            order.
        problem = pulp.LpProblem("racao", pulp.LpMinimize)
        xs = [pulp.LpVariable(f"x{j:04d}", lowBound=0.0) for j in range(n)]
        problem += pulp.lpSum(float(c[j]) * xs[j] for j in range(n) if c[j])
        for i in range(len(b)):
            row = pulp.lpSum(float(A[i, j]) * xs[j] for j in range(n) if A[i, j])
            problem += pulp.LpConstraint(row, PULP_SENSES[senses[i]], f"r{i:04d}",
                float(b_shift[i]))
        with np.errstate(all="ignore"):
            status = PULP_STATUS.get(problem.solve(self._solver()), "
                numerical_error")
        highs = problem.solverModel
        pivots = int(highs.getInfo().simplex_iteration_count)
```

2.3 Integração no RuFaS

O arquivo novo `simplex_optimizer.py` tem o resolvedor de PL, a PLS e a exportação do tableau. Em `RUFAS/biophysical/animal/ration/ration_optimizer.py`, onde o RuFaS chama o `scipy.optimize.minimize`, entrou um

`if/else` (Código 3): com `simplex`, o mesmo objetivo, restrições e limites vão para `_minimize_with_simplex`; senão a chamada original roda sem mudança. Os dois devolvem um objeto com os mesmos campos (`x`, `success`, `fun`). Em `RUFAS/main.py` entrou a opção `-r/-ration-optimizer` (`SLSQP`, padrão, ou `simplex`).

Código 3 Escolha do resolvedor em `attempt_optimization` (`ration_optimizer.py`).

```
        selected_method = (method or RationManager.optimizer_method or "SLSQP").
            lower()
        if selected_method == SIMPLEX_METHOD_NAME:
            optimized_ration_attempt = self._minimize_with_simplex(
                ration_config=ration_config,
                constraints=constraints_to_use,
                bounds=bounds,
                previous_ration=sorted_previous_ration,
                user_defined_ration_dictionary=user_defined_ration_dictionary,
                pen_label=pen_label or (animal_combination.name if
                    animal_combination else "PEN"),
            )
        else:
            optimized_ration_attempt = minimize(
                self.objective,
                initial_decision_vector,
                method="SLSQP",
                bounds=bounds,
                constraints=constraints_to_use,
                args=ration_config,
            )
```

2.4 Como executar

Da raiz do repositório, com o ambiente virtual criado (`pip install .`, Python 3.12 ou 3.13), os comandos do Código 4 rodam a mesma fazenda com os mesmos arquivos e semente; cada rodada leva dois a três minutos. Só a rodada com `-r simplex` cria `output/simplex/GO_campo_dourado/`, com o tableau e o modelo de cada formulação (`tableau_*.csv` e `modelo_*.csv`), e acrescenta ao CSV de dieta as colunas de status, custo Z, iterações, pivôs, preços-sombra e custos reduzidos.

Código 4 Execução com o SLSQP (padrão), com o Simplex e dos testes, da raiz do fork.

```
# resolvedor original do RuFaS (SLSQP)
venv/bin/python main.py -g -p GO_campo_dourado/_pipeline_task_manager_metadata.json

# mesma fazenda, mesmos arquivos, dieta formulada pelo Simplex
venv/bin/python main.py -g -r simplex -p GO_campo_dourado/_pipeline_task_manager_metadata.json

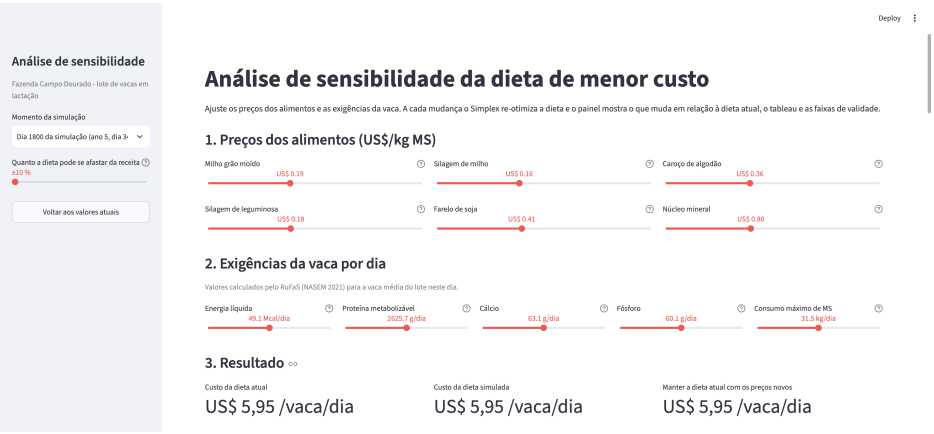
# testes do resolvedor de PL, da linearizacao e da opcao de comando
venv/bin/python -m pytest tests/test_biophysical/test_animal/test_ration/ tests/test_main.py
```

3 ANÁLISE DE SENSIBILIDADE

3.1 Painel

A pasta `analise_sensibilidade/` do fork tem um painel Streamlit (Streamlit, 2026) (`app.py`) e um módulo (`sensibilidade.py`, de onde vêm os Códigos 6 e 7) que lê o tableau e o modelo gravados pelo Simplex, calcula as faixas de validade de c_j e b_i e, fora da faixa, resolve o PL de novo com o mesmo resolvedor do RuFaS. A Figura 1 mostra as primeiras seções do painel.

Figura 1 – *Painel de análise de sensibilidade: preços, exigências e resultado (formulação do dia 1800).*



Fonte: Elaborado pelo autor.

3.2 Como os valores são obtidos

Preços-sombra (variáveis duais) e custos reduzidos. Vêm do próprio Simplex, logo após a solução ótima (Código 5): `row_dual` é $y_i = \partial Z/\partial b_i$ e `col_dual` é $\bar{c}_j = c_j - c_B B^{-1} a_j$; a base ótima (`getBasis`) é o que permite reconstruir o tableau.

Código 5 Leitura dos preços-sombra, custos reduzidos e base ótima (`simplex_optimizer.py`).

```
1        solution, basis_status = highs.getSolution(), highs.getBasis()
2        x = lower + np.asarray(list(solution.col_value), dtype=float)
3        shadow = np.asarray(list(solution.row_dual), dtype=float)  # dZ/db_i in
             each row's own sense
4        reduced = np.asarray(list(solution.col_dual), dtype=float)
5        basic_cols = [j for j, s in enumerate(basis_status.col_status) if s ==
             highspy.HighsBasisStatus.kBasic]
6        basic_rows = [i for i, s in enumerate(basis_status.row_status) if s ==
             highspy.HighsBasisStatus.kBasic]
```

Intervalo de variação de c_j. Alimento não básico: c_j sobe sem limite e cai até $c_j - \bar{c}_j$. Alimento básico: razões entre a linha Z e a linha do alimento nas colunas não básicas, $c_j + \max\{\bar{c}_k / t_{jk} : t_{jk} < 0\} \le c_j \le c_j + \min\{\bar{c}_k / t_{jk} : t_{jk} > 0\}$ (Código 6).

Código 6 Intervalo de c_j a partir do tableau (`sensibilidade.py`).

```
def faixa_custos(tableau: pd.DataFrame, modelo: Modelo) -> pd.DataFrame:
    z = tableau.loc["Z"]
    basicas = [r for r in tableau.index if r != "Z"]
    nao_basicas = [c for c in tableau.columns if c != "RHS" and c not in basicas]
    linhas = []
    for feed in modelo.feeds:
        c_j = float(modelo.variaveis.loc["custo_c", feed])
        x_j = float(modelo.variaveis.loc["x_otimo", feed])
        reduzido = float(z[feed])
        if feed in basicas:
            row = tableau.loc[feed, nao_basicas]
            pos = [z[k] / row[k] for k in nao_basicas if row[k] > EPS]
            neg = [z[k] / row[k] for k in nao_basicas if row[k] < -EPS]
            c_min = c_j + (max(neg) if neg else -np.inf)
            c_max = c_j + (min(pos) if pos else np.inf)
            situacao = "básica"
        else:
            c_min, c_max, situacao = c_j - reduzido, np.inf, "não básica (no limite
                )"
```

Intervalo de variação de b_i. A coluna da folga ou excesso da restrição i no tableau é $\pm B^{-1} e_i$; a base continua viável enquanto $\mathrm{RHS} + \Delta b_i \, B^{-1} e_i \ge 0$, o que dá a faixa pelas razões entre a coluna RHS e essa coluna (Código 7). Dentro dela Z varia $y_i \, \Delta b_i$.

Código 7 Intervalo de b_i a partir do tableau (`sensibilidade.py`).

```
def faixa_rhs(tableau: pd.DataFrame, modelo: Modelo) -> pd.DataFrame:
    basicas = [r for r in tableau.index if r != "Z"]
    rhs = tableau.loc[basicas, "RHS"].to_numpy(dtype=float)
    linhas = []
    for name, row in modelo.restricoes.iterrows():
        sinal, b, y = str(row["sinal"]), float(row["b"]), float(row["preco_sombra
            "])
        coluna = ("folga_" if sinal == LESS_EQUAL else "excesso_") + str(name)
        if coluna not in tableau.columns:
            continue
        d = tableau.loc[basicas, coluna].to_numpy(dtype=float)
        g = d if sinal == LESS_EQUAL else -d
        sobe = (-rhs[g < -EPS] / g[g < -EPS]).min() if np.any(g < -EPS) else np.inf
        desce = (-rhs[g > EPS] / g[g > EPS]).max() if np.any(g > EPS) else -np.inf
        b_min, b_max = b + desce, b + sobe
```

3.3 Formulação do dia 45

No dia 45, primeira formulação em regime, $Z^* =$ US$ 6,1516 por vaca por dia, com 2 pivôs e 2 iterações. A silagem de milho é a única variável básica entre os alimentos (8,089 kg); os outros cinco estão no piso da receita (Tabelas 3 e 4).

Tabela 3 – *Preços-sombra e intervalo do lado direito, dia 45.*

Restrição	b_i	valor em x^*	folga	y_i	b_i mín.	b_i máx.
R1 energia (Mcal)	50,98	50,98	0	0,0814	49,40	52,58
R2 proteína mín. (g)	2 727	2 845	118	0	$-\infty$	2 845
R3 proteína máx. (g)	4 016	2 845	1 171	0	2 845	$+\infty$
R4 cálcio (g)	66,6	141,8	75,2	0	$-\infty$	141,8
R5 fósforo (g)	62,8	77,6	14,8	0	$-\infty$	77,6
R10 consumo mín. (kg)	17,17	25,09	7,92	0	$-\infty$	25,09
R11 consumo máx. (kg)	31,89	25,09	6,80	0	25,09	$+\infty$

Fonte: `modelo_LAC_COW_dia00045_tent1.csv`; R6 a R9 têm folga e $y = 0$.

Tabela 4 – *Custos reduzidos e intervalo dos coeficientes da função objetivo, dia 45.*

Alimento	x_j^* (kg MS)	c_j	custo reduzido	c_j mín.	c_j máx.
Silagem de milho (básica)	8,089	0,16	0	0,00	0,1724
Silagem de leguminosa	4,372	0,18	0,0346	0,1454	$+\infty$
Milho grão	5,829	0,19	0,0137	0,1763	$+\infty$
Farelo de soja	3,886	0,41	0,1983	0,2117	$+\infty$
Caroço de algodão	2,186	0,36	0,1831	0,1769	$+\infty$
Núcleo mineral	0,729	0,80	0,8206	0,00	$+\infty$

Fonte: `tableau_LAC_COW_dia00045_tent1.csv`. O mineral tem energia líquida negativa no modelo linearizado ($-0{,}25$ Mcal/kg), por isso o custo reduzido supera o preço.

O preço-sombra da energia, US$ 0,0814 por Mcal, é o preço da silagem de milho dividido pela sua energia (0,16 / 1,9647). Conferi as faixas re-otimizando o PL (Tabela 5): dentro da faixa de b_1 o custo previsto por $Z^* + 0{,}0814\,\Delta b_1$ coincide com o re-otimizado; fora dela o milho grão entra na base. Para c_1, até US$ 0,1724 a dieta não muda; a US$ 0,18 o milho grão substitui parte da silagem.

Tabela 5 – *Verificação das faixas por re-otimização, dia 45.*

Parâmetro	valor novo	Z previsto	Z re-otimizado	base
b_1 energia	49,5 Mcal	6,0310	6,0310	igual
b_1 energia	53,0 Mcal	6,3161	6,3187	muda (x_3 entra)
c_1 silagem de milho	0,17	6,2325	6,2325	igual
c_1 silagem de milho	0,18	6,3134	6,3073	muda (x_1 7,286; x_3 6,558)
c_3 milho grão	0,30	6,7928	6,7928	igual

Fonte: Calculado por `sensibilidade.py` e conferido com o resolvedor do RuFaS.

Cenário e horizonte. Com o milho grão a US$ 0,30 (+60 %) e a receita presa a ±10 %, a dieta não muda (o milho já está no piso) e o custo sobe de 6,1516 para 6,7928; com a receita a ±30 %, o Simplex reduz o milho para 4,53 kg, eleva as silagens e o custo fica em 6,2723: a flexibilidade vale US$ 0,52 por vaca por dia. Nas 41 formulações a energia foi a restrição ativa dos dias 45 a 1755 (y entre US$ 0,081 e 0,088 por Mcal); no dia 0 foi a proteína mínima, no dia 630 energia e proteína juntas, e o dia 1800 é degenerado: nenhuma exigência ativa, todos os alimentos no piso da receita, preços-sombra nulos e faixa de c_j de zero a infinito. É o dia da Figura 1.

4 RESULTADOS: SIMPLEX VERSUS O OTIMIZADOR EXISTENTE

4.1 Caso base: fazenda receita a ±10 %

Rodei a fazenda Campo Dourado por cinco anos (semente 44) com os dois comandos do Código 4. A Tabela 6 resume o lote de lactação, média das 41 formulações do CSV de dieta de cada rodada; o custo é a dieta praticada vezes os preços da Tabela 1.

Tabela 6 – *Lote de lactação, média das 41 formulações de cada rodada.*

	SLSQP (padrão)	Simplex (`-r simplex`)
Custo da dieta (US$/vaca/dia)	6,12	6,15
Faixa do custo nas 41 formulações	5,88 a 6,40	5,89 a 6,52
Consumo de MS (kg/vaca/dia)	24,93	25,07
Dieta, % da MS (sil. milho / sil. legum. / milho)	31,8 / 17,5 / 23,4	32,1 / 17,5 / 23,2
Dieta, % da MS (f. soja / caroço / mineral)	15,6 / 8,7 / 2,9	15,5 / 8,7 / 2,9
Energia líquida entregue / exigida (Mcal)	45,9 / 45,7	46,1 / 46,0
Proteína metab. entregue / exigida (g)	2 683 / 2 591	2 696 / 2 611
Leite (kg/vaca/dia)	40,10	40,47
Corte de leite por falha na formulação	nenhum	nenhum
Formulações ótimas na primeira tentativa	–	41 de 41
Iterações da linearização / pivôs por formulação	–	2,0 / 2,2

Fonte: CSVs `GO_campo_dourado_saved_variables_csv_dieta.txt_27-Oct-2026_*.csv`.

Os dois resolvedores chegam praticamente à mesma dieta. A diferença de US$ 0,03 por vaca por dia (0,5 %) é menor que a de 1 a 2 % entre duas rodadas do próprio RuFaS com o SLSQP e a mesma semente (o ponto de partida do SLSQP não é semeado).

4.2 Efeito econômico no rebanho

O RuFaS não tem preço de leite para o Brasil; usei a tabela do próprio simulador (`input/data/EEE/econ`, série `milk_all`, linha 01: US$ 0,391, 0,371, 0,402, 0,370 e 0,423 por litro em 2015 a 2019), com 1 kg de leite como 1 litro. A receita é o leite diário do rebanho vezes o preço do ano; o custo é a dieta vigente vezes as vacas em lactação, somado dia a dia (Tabela 7).

Tabela 7 – *Rebanho em lactação ao longo dos cinco anos de cada rodada.*

	SLSQP	Simplex
Vacas em lactação, média (mín. a máx.)	89,3 (78 a 102)	88,1 (76 a 97)
Leite do rebanho (kg/dia)	3 580	3 567
Leite em 5 anos (t)	6 538	6 514
Custo da ração do lote (US$)	994 217	958 659
Receita do leite (US$)	2 556 526	2 545 973
Margem leite menos ração (US$)	1 562 309	1 587 314
Custo da ração por kg de leite (US$)	0,152	0,147
Margem por vaca por dia (US$)	9,58	9,86

Fonte: CSVs das duas rodadas e tabela de preços do RuFaS.

No total, a rodada do Simplex gastou 3,6 % menos com ração e teve margem 1,6 % maior, mas o ganho não vem do solver: por vaca e por dia o Simplex custou US$ 0,03 a mais. A diferença vem de a rodada do Simplex ter tido 1,2 vaca em lactação a menos e 24 t de leite a menos em cinco anos, variação do próprio simulador entre rodadas. Minha conclusão é que, com a receita presa a ±10 %, trocar o solver não muda o resultado econômico; o efeito que importa é o da dieta livre, a seguir.

4.3 Comparação com os mesmos dados: dieta livre

Para comparar os resolvedores sem a interferência do rebanho, o Simplex rodou a simulação com dieta livre (`user_input = false`, em cópia temporária dos arquivos) e, em cada formulação, o SLSQP foi executado sobre os mesmos dados, com o ponto de partida do RuFaS, partindo da solução do Simplex e com dez pontos aleatórios. O resultado está em `COMPARACAO_SOLVERS_LADO_A_LADO.csv`, na raiz do fork, e na Tabela 8.

Tabela 8 – *Simplex e SLSQP sobre os mesmos dados em cada formulação, dieta livre.*

Lote	Simplex	SLSQP	SLSQP − Simplex
Novilhas (41 form.)	41 ótimas; 1,599	41 ok; 1,599	zero em todas
Pré-parto (41 form.)	41 ótimas; 2,405	41 ok; 2,405	zero em todas
Lactação (41 form., 43 tent.)	41 ótimas; 5,23	42 ok; 5,24	+0,006; 6 diferem (máx. 0,045)

Fonte: `COMPARACAO_SOLVERS_LADO_A_LADO.csv`; custos em US$ por animal por dia.

Nas novilhas e no pré-parto as dietas são idênticas. Na lactação, 35 das 41 formulações coincidem; nas outras cada método para em um ótimo local diferente, porque o problema não é convexo, e o SLSQP partindo da solução do Simplex não melhora o custo (máximo US$ 0,016). Dos 430 pontos aleatórios do SLSQP, 422 convergiram; o tempo por formulação foi de cerca de 0,02 s nos dois. No dia 585 a linearização parou em uma dieta com 24 kg de silagem de leguminosa que violava a energia original, e o RuFaS precisou de três tentativas até o Simplex fechar, enquanto o SLSQP fechou na primeira.

A dieta livre custou US$ 5,23 por vaca por dia contra 6,15 da receita do produtor, 15 % a menos, trocando silagem de milho, caroço e mineral por 17,1 kg de silagem de leguminosa (energia, fósforo e proteína ativas). Com as 88 vacas da rodada, a diferença de US$ 0,92 por vaca por dia equivale a cerca de US$ 29 600 por ano em ração. Cabe ao nutricionista julgar se uma dieta com 64 % de silagem de leguminosa é praticável, pois o modelo não tem limite de inclusão por alimento.

5 CONCLUSÃO

O Simplex está integrado ao RuFaS como resolvedor alternativo (`-r simplex`), sem mudança no SLSQP nem nos arquivos da fazenda, por programação linear sequencial com cada PL modelado em PuLP.

Sobre o efeito econômico, com a receita presa a ± 10 % os dois solvers chegam à mesma dieta (US$ 6,15 e 6,12 por vaca por dia) e o custo total menor da rodada do Simplex se deve ao número menor de vacas em lactação, não ao solver; o ganho real está na dieta livre, 15 % mais barata (cerca de US$ 29 600 por ano no lote). Na sensibilidade, a energia líquida governa a dieta em quase todo o horizonte e os intervalos de c_j e b_i do tableau foram conferidos por re-otimização.

Como limitações, o problema não é convexo, os dois métodos podem parar em ótimos locais diferentes e no dia 585 da dieta livre a linearização parou em um ponto inviável. Trabalhos futuros: opção de comando para a dieta livre, limites de inclusão por alimento e preço do leite brasileiro.

Referências

COIN-OR Foundation. **PuLP: a linear programming modeler written in Python**. 2026. `https://www.coin-or.org/pulp/`. Acesso em: 28 out. 2026. 2

National Academies of Sciences, Engineering, and Medicine (NASEM). **Nutrient requirements of dairy cattle**. 8. rev.. ed. Washington, DC: The National Academies Press, 2021. 500 p. 2

NOBRE, Igor Coutinho Ferreira. **RuFaS com o otimizador Simplex (fork, ramo `simplex`)**. 2026. `https://github.com/coutinhonobre/RuFaS`. Acesso em: 28 out. 2026. 2

Ruminant Farm Systems. **RuFaS: Ruminant Farm Systems Model**. 2026. `https://github.com/RuminantFarmSystems/RuFaS`. Acesso em: 28 out. 2026. 2

Streamlit. **Streamlit: a faster way to build and share data apps**. 2026. `https://streamlit.io/`. Acesso em: 28 out. 2026. 7